IBM Redbook

Mainframe-Integrated AI:

Building Cognitive Systems on z/OS and zLinux

On-Premises AI Inference with OpenShift on IBM Z

Covers:

- IBM z16 / z17 Telum AI Accelerators
- vLLM on zLinux (s390x)
- pgvector on OpenShift on Z
- CICS / IMS AI Integration Patterns
- RACF Zero-Trust for AI Workloads
- GDPS-Aware DR for AI Stacks
- Agentic Cage: Safe AI Agents on z/OS
- Terraform IaC Landing Zone

2026 Edition

Chapter 1: The Great Re-Centralization

How AI is Pulling Workloads Back to the Mainframe

Cloud made it easy to rent CPUs; AI made it expensive to rent intelligence. When every token is a dollar, the enterprise needs low-latency, high-governance compute next to the data. That sounds suspiciously like something we have had for decades: IBM Z.

Modern Z systems (z16/z17, LinuxONE 4/5) include on-chip AI inferencing via Telum, designed to run models inline with transactions for use cases like fraud detection, with micro- to low-millisecond latency and without data egress. The cloud promised decentralization, but AI's cost, latency, and governance problems are driving compute back toward centralized, hardened, sovereign platforms.

The mainframe is the original sovereign compute platform. It was not designed for convenience, it was designed for certainty. While probabilistic AI systems introduce new classes of operational risk, the mainframe's architecture provides the governance substrate that enterprises need to contain that risk.

Thesis

- Keep business-critical systems of record on Z.
- Place AI reasoning on Z (z/OS, zCX) or near Z (zLinux / OpenShift).
- Expose strict API contracts to transactional paths.
- Drive everything with Terraform + Git for reproducibility.

> **Architect's Insight**
> *Mainframes were built for trust. AI requires trust. The industry is rediscovering what IBM engineers solved in 1964.*

Cloud-Native AI vs. Mainframe-Integrated AI

Dimension	Cloud-Native AI	Mainframe-Integrated AI
Latency	50-500ms round-trip	<1ms inline (Telum) to 5-50ms (zLinux)
Data egress	Required for inference	Zero egress — model comes to data
Governance	Policy applied externally	RACF, APF, SMF baked into platform
Cost model	Per-token at scale	Predictable MIPS allocation
Compliance	Contractual / attestation	Physical data residency assertion
Availability	Multi-AZ SLA	z/OS RAS — designed for zero downtime

Dimension	Cloud-Native AI	Mainframe-Integrated AI
Audit trail	Logs + SIEM	SMF records — tamper-evident, native

Chapter 2: Understanding the z-Architecture AI Landscape

The Hardware, the OS, and the Hybrid Reality

Before placing any AI workload, an architect must understand the four layers of the z-architecture stack and what each contributes to an AI deployment.

2.1 Telum: The AI Accelerator Built into the Chip

IBM Telum (z16) introduced an integrated AI accelerator, a matrix math engine, directly on the processor chip. This was not a discrete GPU attached via PCIe; it was silicon designed from the start to perform inference inside the same physical die as the CPU executing z/OS transactions. The result is sub-millisecond inferencing latency with no memory copy overhead and no network hop.

Telum II (z17, LinuxONE 5) extends this with updated caches, DPU-accelerated I/O, and ensemble AI capability via the Spyre expansion module, which enables scaled LLM inferencing on LinuxONE 5 / z17 class systems.

Feature	Telum (z16)	Telum II (z17 / LinuxONE 5)
AI engine	On-chip matrix math	Enhanced on-chip + Spyre expansion
Primary use case	Scoring / fraud detection	Scoring + scaled LLM inferencing
Latency profile	Sub-millisecond	Sub-millisecond to low-millisecond
LLM capability	Limited (scoring models)	7B+ class with Spyre
Linux AI toolkit	AI Toolkit for Z (TF/PyTorch/ONNX)	AI Toolkit + Spyre runtime

2.2 z/OS: The Transaction Substrate

z/OS provides the CICS, IMS, and Db2 subsystems where business-critical transaction logic lives. AI integration with z/OS means injecting intelligence into these transaction paths while preserving the RAS (Reliability, Availability, Serviceability) guarantees that z/OS was built to deliver. Key integration surfaces include z/OSMF, SMF, RACF, and z/OS Connect EE.

2.3 zLinux (s390x): The AI Runtime Substrate

Red Hat OpenShift on IBM Z & LinuxONE runs natively on s390x, on LPARs, z/VM guests, or KVM guests. This is the correct home for containerized inference servers (vLLM), vector databases (pgvector), RAG workers, and AI agents. OpenShift on Z supports disconnected installs, making it suitable for air-gapped sovereign deployments.

2.4 zCX: Containers Inside z/OS

z/OS Container Extensions (zCX) allow OCI containers to execute within z/OS address spaces. This is the option when raw PII must be accessible to a model and cannot transit any network boundary, even an internal one. Container workloads in zCX operate within the z/OS security perimeter and are governed by RACF.

2.5 Coupling Facility: The Sysplex Memory Bus

The Coupling Facility provides shared memory structures accessible to all LPARs in a sysplex at memory speed. For AI workloads, the CF becomes a cognitive state bus: prompt dispatch queues (list structures), model version locks (lock structures), and hot RAG result caches (cache structures) can all live in the CF, shared across CICS regions and OpenShift nodes simultaneously.

Architect's Insight

Put high-value inference where your data gravity and governance already live.

Chapter 3: AI on z/OS vs. AI Near z/OS

A Decision Framework

Every AI workload on or near a mainframe must be evaluated across five dimensions before a placement decision is made. Getting any one of them wrong produces either a system that is technically correct but operationally impossible, or one that violates the compliance contracts the mainframe was installed to enforce.

3.1 The Decision Matrix

Dimension	On z/OS	zCX (on z/OS)	zLinux / OCP	Hybrid
Latency	<1ms	1-5ms	5-50ms	Tiered by tier
Model size	Scoring only (no LLM)	Small (<3B)	Medium-Large (7B-70B)	Split by size
Data gravity	PII in z/OS datasets	PII in z/OS datasets	Derived/enriched only	Masking gateway required
Transaction path	Inline (CICS/IMS)	Near-inline	Async / batch	Sync+async
Model type	ONNX scoring, rule engine	Small ONNX / quantized	Full LLM, RAG, agents	Scoring on Z, LLM near Z
Compliance	Must stay on LPAR	Must stay on LPAR	Controlled egress only	Data never leaves sysplex
Acceleration	Telum matrix engine	Telum (via host)	Telum (AI Toolkit)	Both
Tooling	COBOL, PL/I, Assembler	OCI containers on z/OS	OpenShift, Terraform, vLLM	z/OS Connect EE as bridge
Ops model	z/OS systems programmers	z/OS systems programmers	Platform/SRE team	Joint model required
Failure mode	Deterministic fallback in CICS	Contained blast radius	Isolated from z/OS SLA	Gateway absorbs AI failures

3.2 The Five Dimensions of Placement

Latency is the first filter. The Telum matrix engine was designed for sub-millisecond inferencing inside a transaction path, fraud scoring during a payment authorization is the canonical example. That window is simply too tight for a network hop to zLinux, let alone a round-trip through an OpenShift Route. If your SLA is measured in microseconds, the model must live on z/OS and must be a scoring model, not an LLM.

Model size is the second filter, and it immediately eliminates LLMs from the z/OS path. A 7B parameter model at float32 requires roughly 28 GB of addressable memory before KV cache. z/OS address spaces are not the right home for this. zLinux with OpenShift, where Linux memory management and container resource limits apply naturally, is the correct substrate for any full-scale LLM deployment.

Data gravity is the third and most legally consequential filter. If your inference requires raw PII, account numbers, social security numbers, protected health information, that data cannot transit a network boundary to reach a model, even an on-premises one, without explicit data handling controls. When raw PII must reach the model, the model must come to the data: either on z/OS via the AI Toolkit scoring runtime, or inside zCX where OCI containers execute within the z/OS security perimeter.

Transaction path determines synchrony. CICS and IMS transactions are synchronous by nature. Any AI call in the hot path must return within the transaction timeout window. Asynchronous workloads, nightly batch enrichment, document summarization, agent-driven runbook analysis, belong on zLinux where they can queue, retry, and scale independently.

Compliance boundary is the final filter and the one most often underestimated by architects from cloud-native backgrounds. A sysplex is an auditable, physically bounded trust domain. Regulators in financial services, healthcare, and government frequently define compliance in terms of physical data residency. Disconnected OpenShift on Z, with no egress route to any external network, can satisfy requirements that no hyperscaler private cloud offering can match.

3.3 The z/OS Connect EE Bridge

When the decision matrix produces a split result, scoring on z/OS, LLM reasoning on zLinux, z/OS Connect EE is the architectural joint between the two halves. In provider mode it exposes CICS transactions, IMS programs, and Db2 stored procedures as REST/JSON APIs consumable by zLinux workloads. In requester mode it generates COBOL or PL/I stubs from an OpenAPI definition, allowing CICS/IMS programs to call a zLinux REST service as if it were a local subroutine. API mediation enforces TLS, OAuth token validation, JSON schema validation, and rate limiting, the LLM never touches raw COMMAREA data.

Architect's Insight

The question is never 'can we connect z/OS to an LLM?' CICS WEB will happily POST to any URL you give it. The question is whether that data should cross that boundary, and whether you can prove it to your regulator. The matrix gives you the answer before you write a single line of code.

Anti-Pattern

Routing CICS transactions directly to a cloud LLM endpoint via an HTTP URIMAP. This causes PII egress, unpredictable token costs, cloud provider SLA misalignment with z/OS RAS requirements, and an audit trail that stops at the firewall.

Chapter 4: Building AI-Enhanced CICS & IMS Applications

Injecting Intelligence Into Transaction Systems

AI can sit 'in the path' of mission-critical mainframe workflows. The key architectural principle is that AI augments deterministic logic, it does not replace it. Every AI call in a CICS pipeline must have a deterministic fallback, a validation gate on the response, and a timeout that protects the transaction SLA.

4.1 CICS Inline Guardrail Pattern

The following sequence diagram shows the recommended flow for AI-augmented CICS transactions:

```
sequenceDiagram
  autonumber
  participant U as User/Channel
  participant C as CICS Txn
  participant V as Vector DB (pgvector)
  participant M as Model Svc (OpenShift on Z)
  Note over C: URIMAP → SSO/TLS policy
  U->>C: Transaction request (JSON)
  C->>C: Deterministic checks
  C->>M: EXEC CICS WEB POST /v1/chat/completions
  M->>V: semantic query (tenant namespace)
  V-->>M: top-k chunks
  M-->>C: answer + evidence
  C->>C: validate → threshold + schema
  C-->>U: decision (approve/deny/explain)
```

4.2 Integration Patterns

- Use z/OS Connect EE when you need API mediation and JSON mapping to COMMAREAs, channels/containers.
- Use EXEC CICS WEB when you want a tight, low-overhead HTTP client in the task.
- Use z/OS Connect requester mode to generate COBOL/PL/I stubs from OpenAPI specs, no manual HTTP coding.

4.3 Anti-Patterns

Anti-Pattern 1

Shipping raw PII to a public LLM, never bypass z/OS data masking or RACF role checks.

Anti-Pattern 2

Letting AI decide transaction state without a deterministic fallback. The model answer is advisory; CICS logic is authoritative.

Anti-Pattern 3

Tying transaction SLA to a fragile external API. All AI calls must have timeout handling and a safe default response.

Chapter 5: Secure On-Prem Model Hosting on zLinux

Building Your Mainframe-Native AI Stack

This chapter builds the containerized AI inference stack on zLinux using OpenShift on IBM Z. The primary inference runtime is vLLM, chosen for its OpenAI-compatible API, production-grade throughput characteristics, and CPU-only build path for s390x.

5.1 OpenShift on Z: Your AI Substrate

OpenShift runs natively on IBM Z & LinuxONE (LPAR, z/VM, KVM), including disconnected installs, and is the right control plane for containerized inference, RAG workers, and agent tools. Treat the OpenShift cluster as your AI landing zone with strict Namespaces, NetworkPolicies, and service accounts mapped to RACF personas.

5.2 Why vLLM

vLLM provides an OpenAI-compatible REST API (/v1/chat/completions, /v1/completions), making it the natural inference server for any client that can call a REST endpoint, including CICS via EXEC CICS WEB. On s390x the build uses CPU-only PyTorch (no CUDA, no ROCm). For Telum-accelerated inference, IBM's AI Toolkit for Z provides optimized TensorFlow, PyTorch, and ONNX runtimes that exploit the Telum matrix engine on z16/z17.

5.3 Namespace Layout

Namespace	Purpose	Key workloads
ai-inference	Model serving	vLLM deployment, model PVC
ai-ingest	RAG ingestion pipeline	pgvector StatefulSet, embedding jobs
ai-registry	Prompt and model registry	Git-synced ConfigMaps, version tracking
ai-observability	Metrics and dashboards	Grafana, Prometheus, SMF bridge
ai-agents	Agent runtime	agent-tool-catalog deployment

5.4 Network Segmentation

Network isolation between the AI namespaces and the rest of the cluster is enforced through OpenShift NetworkPolicy objects. The fundamental principle is zero implicit trust: no namespace can reach another unless a policy explicitly permits it, and no namespace can reach z/OS subsystems directly, all z/OS traffic is routed through the z/OS Connect EE gateway.

NetworkPolicy: ai-inference (vLLM)

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: ai-inference-policy
  namespace: ai-inference
spec:
  podSelector: {}       # applies to all pods in namespace
  policyTypes:
    - Ingress
    - Egress
  ingress:
    # Only ai-ingest (RAG queries) and ai-agents may call vLLM
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: ai-ingest
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: ai-agents
      ports:
        - port: 8000
          protocol: TCP
    # z/OS Connect EE gateway (CICS caller) reaches vLLM via this port
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: zosmf-gateway
      ports:
        - port: 8000
          protocol: TCP
  egress:
    # vLLM may only reach ai-ingest (pgvector for RAG)
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: ai-ingest
      ports:
        - port: 5432
          protocol: TCP
    # DNS resolution must be permitted
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
      ports:
        - port: 53
          protocol: UDP
```

NetworkPolicy: ai-ingest (pgvector)

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: ai-ingest-policy
  namespace: ai-ingest
spec:
  podSelector: {}
```

```
  policyTypes:
    - Ingress
    - Egress
  ingress:
    # Only ai-inference (vLLM RAG queries) and batch ingest jobs may connect
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: ai-inference
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: ai-ingest   # intra-namespace batch jobs
      ports:
        - port: 5432
          protocol: TCP
  egress:
    # pgvector has no outbound egress — it is a data store only
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
      ports:
        - port: 53
          protocol: UDP
```

NetworkPolicy: ai-agents

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: ai-agents-policy
  namespace: ai-agents
spec:
  podSelector: {}
  policyTypes:
    - Ingress
    - Egress
  ingress:
    # Agents are called only from the ops gateway (authenticated operators)
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: ops-gateway
      ports:
        - port: 8080
          protocol: TCP
  egress:
    # Agents may call vLLM for reasoning
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: ai-inference
      ports:
        - port: 8000
    # Agents may call z/OSMF (tool execution) — IP-restricted to sysplex
    - to:
        - ipBlock:
            cidr: "10.0.0.0/8"     # Replace with your sysplex CIDR
            except:
```

```
            - "10.0.0.0/24"      # Exclude non-z/OSMF ranges as needed
      ports:
        - port: 443
          protocol: TCP
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
      ports:
        - port: 53
          protocol: UDP
```

Network Policy Note

These policies assume OVN-Kubernetes or OpenShift SDN with NetworkPolicy enforcement enabled. Verify with: oc get network.config/cluster -o jsonpath='{.spec.networkType}'. OVN-Kubernetes is recommended for new deployments on IBM Z.

5.5 Kerberos Integration

In enterprise mainframe environments, Kerberos (via IBM's RACF Kerberos support) is the preferred authentication mechanism for service-to-service calls between z/OS and zLinux workloads. It eliminates password-based credentials from the API path and integrates cleanly with RACF group memberships.

Integration Architecture

Component	Kerberos Role	Principal format
z/OS RACF	KDC (Key Distribution Center)	user@MYREALM.LOCAL
z/OS Connect EE	Service principal (verifies tokens)	HTTP/zosc.sysplex.local@MYREALM.LOCAL
OpenShift ServiceAccount	Client principal (obtains TGT)	aiapp/ocp.sysplex.local@MYREALM.LOCAL
vLLM service	No Kerberos — internal cluster only	OCP ServiceAccount token
AI Agent	Client principal for z/OSMF calls	agent/ocp.sysplex.local@MYREALM.LOCAL

Configuring a Kerberos Client in an OpenShift Pod

```
# krb5.conf — mount as ConfigMap into agent and gateway pods
[libdefaults]
  default_realm = MYREALM.LOCAL
  dns_lookup_realm = false
  dns_lookup_kdc = false
  ticket_lifetime = 8h
```

```
  renew_lifetime = 7d
  forwardable = true

[realms]
  MYREALM.LOCAL = {
    kdc = racf-kdc.sysplex.local:88
    admin_server = racf-kdc.sysplex.local:749
  }

[domain_realm]
  .sysplex.local = MYREALM.LOCAL
  sysplex.local = MYREALM.LOCAL
```

```
# Kubernetes Secret — keytab for the agent service principal
# Generate on z/OS with: KERBEROS ADD KEYTAB(agent/ocp.sysplex.local)
# Then base64-encode and store as a Secret:

apiVersion: v1
kind: Secret
metadata:
  name: agent-keytab
  namespace: ai-agents
type: Opaque
data:
  agent.keytab: <base64-encoded keytab>
```

```
# agent Deployment — mount krb5.conf and keytab
spec:
  containers:
    - name: agent
      volumeMounts:
        - name: krb5-conf
          mountPath: /etc/krb5.conf
          subPath: krb5.conf
          readOnly: true
        - name: agent-keytab
          mountPath: /etc/security/keytabs
          readOnly: true
      env:
        - name: KRB5_CONFIG
          value: /etc/krb5.conf
        - name: KRB5CCNAME
          value: /tmp/krb5cc_agent   # ticket cache in ephemeral storage
  initContainers:
    # Obtain TGT at pod startup before the agent container starts
    - name: kinit
      image: registry.access.redhat.com/ubi9/ubi:latest
      command:
        - /bin/bash
        - -c
        - kinit -kt /etc/security/keytabs/agent.keytab
agent/ocp.sysplex.local@MYREALM.LOCAL
      volumeMounts:
        - name: krb5-conf
          mountPath: /etc/krb5.conf
          subPath: krb5.conf
        - name: agent-keytab
```

```
          mountPath: /etc/security/keytabs
  volumes:
    - name: krb5-conf
      configMap:
        name: krb5-config
    - name: agent-keytab
      secret:
        secretName: agent-keytab
```

Ticket Renewal

Kerberos tickets expire. For long-running pods, add a sidecar container that runs kinit -R on a cron schedule (every 6 hours for an 8-hour ticket lifetime with a 7-day renewal window). Alternatively, configure the Kerberos client library with ticket renewal enabled and let it manage the renewal automatically. Do not embed plaintext passwords in pod specs as a workaround, this defeats the purpose of Kerberos.

```
# Ticket renewal sidecar (add alongside the agent container)
- name: krb-renew
  image: registry.access.redhat.com/ubi9/ubi:latest
  command:
    - /bin/bash
    - -c
    - while true; do sleep 21600; kinit -R; done
  env:
    - name: KRB5_CONFIG
      value: /etc/krb5.conf
    - name: KRB5CCNAME
      value: /tmp/krb5cc_agent
  volumeMounts:
    - name: krb5-conf
      mountPath: /etc/krb5.conf
      subPath: krb5.conf
```

Architect's Insight

Your mainframe becomes the AI Sovereignty Core, where your business logic, identity, and data gravity live. Kerberos makes that identity claim traversable: the same RACF realm that governs your CICS regions also governs your OpenShift agents.

Chapter 6: The Vector Database Behind the Firewall

RAG on zLinux and Hybrid Replication

Retrieval-Augmented Generation (RAG) requires a vector database for semantic search. On zLinux, the recommended choice is Postgres with the pgvector extension. Postgres is proven on s390x; pgvector adds approximate nearest-neighbor (ANN) indexing with fewer multi-architecture surprises than specialized vector databases.

6.1 pgvector Design Checklist

- Dimensions pinned to embedding model output (e.g., 1536 for text-embedding-3-small)
- Tenant isolation through schema or namespace
- Encryption in transit (TLS) and at rest (LUKS on PVC)
- Disaster recovery: WAL streaming replica to secondary LPAR (see Chapter 10)
- IVFFlat or HNSW index, choose based on recall vs. build-time trade-off

6.2 Schema Design for Multi-Tenant RAG

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE IF NOT EXISTS docs (
  id        bigserial PRIMARY KEY,
  tenant    text NOT NULL,
  doc_id    text NOT NULL,
  embedding vector(1536),
  content   text,
  meta      jsonb
);

CREATE INDEX IF NOT EXISTS idx_docs_tenant ON docs(tenant);
CREATE INDEX IF NOT EXISTS idx_docs_vec
  ON docs USING ivfflat (embedding vector_cosine_ops);
```

6.3 Build Strategy

Build the pgvector image in-cluster using OpenShift BuildConfig. This avoids multi-arch binary distribution issues, the image is compiled on s390x for s390x. The build installs PostgreSQL 15 from the UBI9 repo, then compiles pgvector from source with pg_config on PATH. See Appendix A.3 for the complete Terraform module.

Chapter 7: Bringing AI to COBOL, JCL, and REXX

The 'Legacy' Layer Becomes the Cognitive Layer

COBOL is not a legacy language on the mainframe, it is the primary transaction language for trillions of dollars of daily commerce. The goal is not to replace COBOL with AI, but to give COBOL programs access to reasoning capabilities they have never had before.

7.1 Integration Surfaces

- Enterprise COBOL JSON PARSE/GENERATE, parse model responses directly in COBOL working storage
- EXEC CICS WEB, in-task HTTP client for calling vLLM from within a CICS transaction
- z/OS Connect requester generated stubs from OpenAPI specs, no manual HTTP coding
- JCL + BPXBATCH orchestrate batch ingestion (curl or Python) from USS
- REXX + USS scriptable AI calls for system automation and operator tooling

7.2 JSON PARSE in Enterprise COBOL

Enterprise COBOL supports JSON PARSE and JSON GENERATE natively. The NAME OF phrase maps dotted JSON paths (including array indexing) to COBOL data names, handling the structural mismatch between JSON and COBOL working storage cleanly. See Appendix B.1 for a complete working example.

7.3 Batch AI Pipelines

JCL steps can call USS programs via BPXBATCH, enabling curl-based ingestion of documents into the RAG pipeline as part of nightly batch runs. This turns existing batch jobs into AI-enabled intelligence pipelines without rewriting any COBOL.

Sidebar: The First Hybrid Workforce

COBOL and Llama working side-by-side. The 60-year-old language and the 7-billion-parameter model. Neither replaces the other.

Chapter 8: Identity, Access & Zero-Trust for Mainframe AI

Mainframe Security Meets Probabilistic Systems

RACF (Resource Access Control Facility) is the identity and authorization backbone of z/OS. Every AI workload model server, agent, RAG pipeline, CICS caller must have a RACF identity with least-privilege access. A hallucinating model with APF authorization is the stuff of nightmares.

8.1 RACF as Your AI Policy Engine

RACF authenticates users via passwords, digital certificates, Kerberos, and PassTickets. It governs roles and groups, logs all access attempts to SMF, and secures datasets and resources. For AI workloads, map every service account and API caller to a named RACF group.

8.2 Identity Mapping Pattern

Layer	Identity	Governed by
z/OS CICS caller	RACF user with CICS SURROGAT	RACF profiles
z/OS Connect EE	RACF user + TLS client cert	RACF + keyring
OpenShift ServiceAccount	Kubernetes SA + RACF-mapped persona	OCP RBAC + gateway
vLLM service	SA with no z/OS access	OCP NetworkPolicy
AI Agent	Least-privilege RACF ID	RACF + OCP Role
Approver (HITL)	Named RACF user(s)	z/OSMF workflow policy

8.3 Zero-Trust Segmentation

- z/OS side: RACF user/group with least privilege, no OPERATIONS, no SPECIAL
- OpenShift side: ServiceAccount bound to namespace-scoped Role only
- NetworkPolicy: ai-inference namespace cannot reach z/OS directly all traffic via z/OS Connect EE gateway
- Prompt budget enforced at gateway layer, no model can consume unbounded tokens

Architect's Warning

A hallucinating model with APF authorization is the stuff of nightmares. Treat model service accounts exactly as you treat privileged batch job IDs with documented, audited, minimal permissions.

Chapter 9: Observability for Mainframe-Integrated AI

AI-Ops Meets SMF

Traditional mainframe observability centers on SMF records, OMEGAMON dashboards, and sysplex-wide metrics. AI workloads add new failure modes that do not appear in any of these: semantic failures that return HTTP 200, RAG cache misses that silently degrade answer quality, and model latency spikes that violate transaction SLAs without generating any z/OS condition code.

9.1 Correlation Chain

The fundamental unit of AI observability is the correlation chain that links a CICS transaction to every AI operation it triggered:

```
Transaction ID  ⟷  Prompt ID  ⟷  Vector Retrieval ID  ⟷  Model Response ID
```

CICS emits SMF type 110 records (monitoring and statistics). Correlate Transaction ID to Prompt ID to Retrieval latency to spot semantic failures that still return HTTP 200. A transaction that completes successfully but returns an AI-generated apology instead of a decision is a semantic failure invisible to traditional monitoring.

9.2 Dashboard Anchors

Metric	Source	Alert threshold
CICS Txn time vs model latency	SMF 110 + vLLM metrics	>2x baseline p95
Vector search p95	pgvector pg_stat_statements	>200ms
RAG hit rate	AI gateway log	<70% hit rate
Apology rate	Semantic failure proxy — keyword match	>2% of responses
Inference errors	vLLM /metrics endpoint	>0 per minute
Token budget consumption	Gateway rate limiter	>80% of budget

9.3 OMEGAMON Integration

Use OMEGAMON for z/OS for system and sysplex views. Wire AI metrics from Prometheus (deployed in ai-observability namespace) into the same pane of glass. Grafana on zLinux provides the visualization layer for AI-specific dashboards that can sit alongside CICS region dashboards.

Chapter 10: Mainframe Sovereignty & Cross-Region Resilience

Building Geo-Resilient AI Without the Cloud

Standard cloud-native DR thinking 'spin up a replica in us-east-2' does not translate to a sovereign mainframe AI deployment. You cannot spin up a second z16 on demand. The resilience architecture must be designed into the platform from day one.

10.1 New AI Failure Domains

The AI stack introduces failure domains that traditional GDPS planning did not need to consider:

Failure Domain	Traditional GDPS Scope	AI Addition
Compute	LPAR failover	vLLM pod rescheduling across LPARs
Storage	DASD replication (XRC/GM)	Postgres WAL + pgvector index replication
State	CF list/lock structures	Prompt registry, model version, RAG session state
Identity	RACF database sync	ServiceAccount to RACF persona mapping
Configuration	Parmlib sync	Terraform state, Kubernetes etcd, OCP operator config

This is the Soul/Brain/Memory/Body principle applied to mainframe DR:

- Soul = Prompt registry (versioned, replicated)
- Brain = Model weights + config (replicated to secondary PVC)
- Memory = pgvector index + Postgres WAL (synchronous replication)
- Body = OpenShift cluster on secondary LPAR (pre-provisioned, warm standby)
- Identity = RACF database + ServiceAccount mappings (synced)

10.2 GDPS Integration

GDPS (Geographically Dispersed Parallel Sysplex) handles z/OS image failover, XRC or Global Mirror for DASD, sysplex timer synchronization, and automatic IPL sequencing. For AI workloads you must additionally ensure: the OpenShift cluster on the DR LPAR is pre-provisioned and warm, model weights are present on the DR PVC, and the WAL stream from primary pgvector is replicating continuously.

10.3 Coupling Facility as AI State Bus

Three CF structure types serve AI workloads:

```
/* List structure — prompt dispatch queue */
IXLCONN STRUCTNAME(AIPROMPTQ)
         CONNAME(AICICS01)
         INITCOUNT(1000)

/* Lock structure — model version gating during rolling updates */
IXLCONN STRUCTNAME(AIMODELLOCK)
         CONNAME(AIVLLM01)

/* Cache structure — hot RAG results shared across CICS regions */
IXLCONN STRUCTNAME(AIRAGCACHE)
         CONNAME(AICICS01)
         INITCOUNT(50000)
```

The RAG cache structure is the highest-value addition. When multiple CICS regions across LPARs handle the same customer segment, the top-k retrieval results for common query patterns can be cached in the CF and shared across all regions, eliminating redundant vector searches and flattening latency spikes during peak transaction volumes.

10.4 Postgres WAL Replication for pgvector

```
-- postgresql.conf (primary)
wal_level = replica
max_wal_senders = 3
wal_keep_size = 1GB
synchronous_standby_names = 'dr-pgvector-01'

-- pg_hba.conf (primary)
host replication replicator dr-lpar-ip/32 md5

-- postgresql.conf (DR replica, Postgres 12+)
primary_conninfo = 'host=primary-pgvector.ai-ingest.svc.cluster.local
                    port=5432 user=replicator
                    application_name=dr-pgvector-01'
restore_command = 'cp /wal_archive/%f %p'
recovery_target_timeline = 'latest'
```

10.5 Model Weight Replication

Model weights (7B float32 = ~28 GB) change infrequently. Use a scheduled rsync CronJob rather than continuous streaming. See Appendix A for the model-replication Terraform module.

Architect's Insight

GDPS will bring your z/OS image back. It will not bring back a hallucinating model that has lost its prompt registry, a pgvector index that is 4 hours stale, or a ServiceAccount whose RACF mapping no longer exists. Plan the AI stack's DR explicitly, it is not covered by your existing RTO/RPO contracts.

Chapter 11: AI Agents on the Mainframe

The First True 'Cognitive Sysprog'

AI agents in cloud-native environments operate in a forgiving failure domain. A misconfigured agent that deletes an S3 bucket causes an incident. A misconfigured agent with access to MVS operator commands, APF-authorized datasets, or JES2 job streams causes a catastrophe. The architecture of a mainframe AI agent is therefore not primarily a question of capability it is a question of containment.

11.1 The Agentic Cage Architecture

Every design decision starts from the assumption that the agent will eventually receive a malformed prompt, a hallucinated tool call, or a deliberately crafted adversarial input. The system must be safe in all three cases. The Agentic Cage provides three containment layers:

Layer	Mechanism	Blocks
Layer 1: Prompt Policy	System instructions in LLM context	Intent-level prohibited actions
Layer 2: OpenAPI Schema	Required fields: rationale, ticket_id	Malformed or incomplete tool calls
Layer 3: Server-side checks	RACF, timeguard, idempotency, approval records	Execution without authorization

11.2 Tool Catalog: Read-Only (No Approval Required)

Tool	z/OS Mechanism	Returns
get_job_status	z/OSMF Jobs REST API	JES2 job status, RC, sysout summary
list_active_address_spaces	z/OSMF Resource Monitoring	Active ASIDs, CPU%, storage
get_dataset_catalog_entry	z/OSMF Data Sets API	VOLSER, DSORG, RECFM, LRECL
get_wlm_policy	z/OSMF WLM API	Active service definition, goals
get_cics_region_stats	CICS CMCI REST API	Task counts, tx rates, DSA usage
get_smf_summary	USS: smfxtract + parse	Last N SMF records for given type
analyze_syslog	z/OSMF System Log API	Filtered syslog matching pattern
validate_jcl	z/OSMF Jobs API (dry run)	Validation result, no submission

11.3 Tool Catalog: Write (HITL Approval Required)

Tool	z/OS Mechanism	Approvers
submit_job	z/OSMF Jobs REST API	SYSPROG + AIOPS
set_wlm_policy	z/OSMF WLM API	WLM admin + SYSPROG
cancel_job	z/OSMF Jobs REST API	AIOPS
update_cics_urimap	CICS CMCI REST API	CICS admin + SYSPROG
trigger_workflow	z/OSMF Workflow API	Workflow owner

11.4 Permanently Blocked Operations

Hard Blocks — Never Routed

DELETE dataset | ALTER APF list | MODIFY started task without approval record | IPL (any form) | VARY OFFLINE any device | SETROPTS changes | RACF ALTUSER with SPECIAL or OPERATIONS | Any command affecting GDPS or XRC replication

11.5 SMF Audit Integration

Every agent action approved or rejected produces an SMF type 110 subtype 99 record. The record includes: timestamp, agent RACF userid, session ID, tool name, plain-English rationale, approver ID (if applicable), outcome (EXECUTED | REJECTED | BLOCKED), and token count. This is non-negotiable for regulated environments. See Appendix A for the smf_writer.py module.

Architect's Warning

The agent is only as trustworthy as its cage. The cage is only as strong as its weakest layer. On a mainframe, the weakest layer is almost always the one someone added to make the demo easier. The blocked_patterns.json is a policy document, not a config file.

Chapter 12: The Platform Blueprint

Mainframe as the AI Heart of the Enterprise

This chapter combines every prior chapter into a single deployable architecture. All components are provisioned by Terraform, versioned in Git, and documented in an AI-BOM. The mainframe becomes the AI governance anchor of the enterprise.

12.1 End-to-End Flow

```
User → API Gateway → z/OS Connect EE → CICS Transaction
                                  ↓
                           EXEC CICS WEB POST
                                  ↓
                        vLLM (OpenShift on Z)
                                  ↓
                     pgvector RAG (semantic search)
                                  ↓
                    Response → Validation Gate → CICS → Commit
                                  ↓
                         SMF 110 Audit Record
```

12.2 Terraform Module Dependency Graph

```
providers.tf
    └── module.pgvector_on_ocp        (Appendix A.3)
    └── module.vllm_inference         (Appendix A.2)
    └── module.zosmf_approval_gate    (Appendix A.4)
            └── module.agent_tool_catalog  (Chapter 11)
                        depends_on: [pgvector, vllm, approval_gate]
```

12.3 Namespace Topology

Namespace	Module	NetworkPolicy
ai-inference	openshift-vllm	Ingress: ai-ingest, CICS gateway only
ai-ingest	pgvector-on-ocp	Ingress: ai-inference only; Egress: none
ai-registry	prompt-registry ConfigMap	ReadOnly from ai-inference, ai-agents
ai-observability	grafana + prometheus	Ingress: ops team only
ai-agents	agent-tool-catalog	Egress: zosmf URL only; no direct z/OS

12.4 AI-BOM: Mainframe Edition

The AI Bill of Materials records every component of the AI stack at a point in time, ready for auditors and change control boards. It is generated by a job that reads Terraform state and the live cluster API:

```
{
  "timestamp": "2026-02-11T12:00:00Z",
  "environment": "prod",
  "sovStack": {
    "platform": "IBM Z / LinuxONE",
    "zHardware": "z16 / z17",
    "telum": "enabled",
    "gdps": "on"
  },
  "model": {
    "name": "vllm-ibm-granite-7b",
    "hash": "sha256:...",
    "version": "2026-01-10"
  },
  "prompt": {
    "system": "/var/zosmf/prompts/prod/cics-guard/v42.txt",
    "versionId": "3a98f..."
  },
  "vector": {
    "engine": "pgvector",
    "dimension": 1536,
    "dsn": "postgresql://.../postgres",
    "schema": "tenant_prod"
  },
  "controls": {
    "identity": "RACF:AIAPP@PROD",
    "policy": "deny-write-ops-without-approval",
    "budget": { "maxTokensPerMin": 5000 }
  },
  "agent": {
    "blocked_patterns_hash": "sha256:...",
    "max_iterations": 10,
    "hitl_required": true
  },
  "logs": {
    "smf": "type110-subtype99",
    "aiGateway": "journaled",
    "retentionDays": 365
  }
}
```

Final Architect's Insight

The mainframe was not made obsolete by AI. It was waiting for it. The only platform on earth designed from silicon to software for zero-downtime trillion-transaction workloads is also the only platform that can govern AI at the same level of rigor. Sovereignty, reliability, and trust are not cloud features. They are mainframe features.

12.5 About the Author

Britley Hoff is a Master Electrician, combat veteran, and systems architect who has spent over two decades mastering the invisible infrastructure that powers the modern world. Her journey with the mainframe began not in a lecture hall, but through the grit of self-directed research and a deep-seated obsession with how systems, both electrical and digital scale under pressure.

From her early contributions to the **openMosix** cluster initiative and **Slackware Linux** to managing massive **zLinux** environments, Britley has always been drawn to the "Big Iron." To her, the mainframe was never a legacy burden; it was the ultimate engineering feat, a platform designed for the kind of absolute reliability that mission-critical environments demand.

The turning point came with the rise of Large Language Models. While the world rushed toward the public cloud, Britley saw a looming crisis of sovereignty. She realized that AI and the mainframe are not opposing forces, but natural partners. If the mainframe is the enterprise's "Source of Truth," then it is the only logical home for the enterprise's "Intelligence." To Britley, **Sovereignty** isn't just a technical spec; it is a fundamental right to ownership. the ability for an institution to run its own intelligence without a middleman, anchored by the institutional trust only a mainframe can provide.

For the **z/OS systems programmer** holding this book and wondering if the world has moved on: it hasn't. Your skills in workload management, I/O bandwidth, and high availability are the exact requirements for the next era of computing. This book is proof that your expertise is the foundation of the **Cognitive Sysplex**.

Britley lives in **South Carolina**, where she balances her time between architecting AI-native transaction systems and managing a fresh-foods hobby farm. She believes the mainframe will remain the governance anchor of the enterprise for the next sixty years, evolving from a processor of records to the heartbeat of sovereign intelligence.

12.5 A Closing note

If there is one lesson to take from these pages, it is that the mainframe was not made obsolete by AI; it was waiting for it. For sixty years, we built the world's most reliable platform for deterministic record-keeping. Now, we are entering the era of the Cognitive Sysplex. Where that same hardware provides the only substrate rigorous enough to handles the risks of probabilistic intelligence.

We have discussed the "how" the Terraform deployments, the Telum matrix engines, and the vLLM runtimes. But the "why" is far more important. In a world where data is increasingly fragmented and intelligence is rented by the token, the ability to run your own models in your own iron is the ultimate expression of Sovereignty.

To the systems programmers and architects who keep the world's most vital engines running: your expertise in RAS(Reliability, Availability, Serviceability) is now the most valuable skill in AI. You are the guardians of the enterprise's truth. By integrating AI into the heart of the mainframe, you arn't just moderenizing a platform; you are ensuring that the governance anchor of the global economy remains rock-solid for the next sixty years.

The weights are loaded, the RACF profiles are set, and the CICS pipelines are ready. It's time to let Big-Iron think!

Appendix A: Terraform Modules

Each module authenticates to OpenShift using the kubernetes and helm providers (cluster-admin or appropriate RBAC). Images for s390x are built in-cluster using OpenShift BuildConfig + ImageStream to avoid multi-arch distribution issues.

A.1 Provider Bootstrap

```
# providers.tf
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    kubernetes = { source = "hashicorp/kubernetes", version = "~> 2.32" }
    helm       = { source = "hashicorp/helm",       version = "~> 2.13" }
    http       = { source = "hashicorp/http",       version = "~> 3.4"  }
    null       = { source = "hashicorp/null",       version = "~> 3.2"  }
  }
}

provider "kubernetes" {
  host                   = var.kube_host
  token                  = var.kube_token
  cluster_ca_certificate = base64decode(var.kube_ca)
}

provider "helm" {
  kubernetes {
    host                   = var.kube_host
    token                  = var.kube_token
    cluster_ca_certificate = base64decode(var.kube_ca)
  }
}
```

A.2 Module: openshift-vllm (vLLM on s390x)

This module builds a vLLM CPU-only image on-cluster and deploys it with liveness/readiness probes and a model PVC.

files/Dockerfile

```
FROM registry.access.redhat.com/ubi9/ubi:latest

RUN dnf -y install \
      python3.11 python3.11-pip python3.11-devel \
      gcc gcc-c++ make git \
    && dnf clean all \
    && useradd -m vllm

USER vllm
```

```
WORKDIR /home/vllm

# vLLM CPU-only build for s390x (no CUDA, no ROCm)
RUN pip3.11 install --user \
      torch --index-url https://download.pytorch.org/whl/cpu \
    && pip3.11 install --user vllm

ENV PATH="/home/vllm/.local/bin:${PATH}"
ENV VLLM_TARGET_DEVICE=cpu

EXPOSE 8000
ENTRYPOINT ["python3.11", "-m", "vllm.entrypoints.openai.api_server", \
            "--model", "/models/model", \
            "--dtype", "float32", \
            "--device", "cpu", \
            "--host", "0.0.0.0", "--port", "8000"]
```

main.tf

```
resource "kubernetes_namespace_v1" "ns" {
  metadata { name = var.namespace }
}

resource "kubernetes_manifest" "imagestream" {
  manifest = yamldecode(templatefile(
    "${path.module}/templates/imagestream.yaml",
    { name = var.image_name, ns = var.namespace }
  ))
}

resource "kubernetes_manifest" "buildconfig" {
  manifest = yamldecode(templatefile(
    "${path.module}/templates/buildconfig.yaml",
    { name = var.image_name, ns = var.namespace }
  ))
}

resource "kubernetes_deployment_v1" "vllm" {
  metadata {
    name      = "vllm-svc"
    namespace = var.namespace
    labels    = { app = "vllm" }
  }
  spec {
    replicas = var.replicas
    selector { match_labels = { app = "vllm" } }
    template {
      metadata { labels = { app = "vllm" } }
      spec {
        container {
          name  = "vllm"
          image = "${var.image_name}:latest"
          port  { container_port = 8000 }
          env { name = "VLLM_TARGET_DEVICE"; value = "cpu" }
          env { name = "HF_HOME";            value = "/models/.cache" }
          resources {
            limits   = { cpu = var.cpu_limit, memory = var.mem_limit }
            requests = { cpu = var.cpu_req,   memory = var.mem_req   }
          }
```

```
          liveness_probe {
            http_get { path = "/health"; port = 8000 }
            initial_delay_seconds = 60; period_seconds = 30
          }
          readiness_probe {
            http_get { path = "/health"; port = 8000 }
            initial_delay_seconds = 90; period_seconds = 15
          }
          volume_mount { name = "models"; mount_path = "/models" }
        }
        volume {
          name = "models"
          persistent_volume_claim {
            claim_name =
kubernetes_persistent_volume_claim_v1.models.metadata[0].name
          }
        }
      }
    }
  }
}

resource "kubernetes_persistent_volume_claim_v1" "models" {
  metadata { name = "vllm-models"; namespace = var.namespace }
  spec {
    access_modes = ["ReadWriteOnce"]
    resources { requests = { storage = var.model_storage } }
  }
}

resource "kubernetes_service_v1" "svc" {
  metadata { name = "vllm"; namespace = var.namespace }
  spec {
    selector = { app = "vllm" }
    port { name = "http"; port = 80; target_port = 8000 }
  }
}

resource "kubernetes_manifest" "route" {
  manifest = yamldecode(templatefile(
    "${path.module}/templates/route.yaml",
    { name = "vllm", ns = var.namespace }
  ))
}

output "inference_url" {
  value       = "https://vllm-${var.namespace}.${var.cluster_apps_domain}"
  description = "OpenAI-compatible: POST /v1/chat/completions"
}
```

A.3 Module: pgvector-on-ocp

Builds Postgres 15 + pgvector in-cluster and deploys as a StatefulSet with WAL replication support.

files/Dockerfile

```
FROM registry.access.redhat.com/ubi9/ubi:latest

RUN dnf -y install \
      postgresql15 postgresql15-server postgresql15-devel \
      make gcc git \
    && dnf clean all

# pgvector requires pg_config on PATH
ENV PATH="/usr/pgsql-15/bin:${PATH}"
ENV PGDATA="/var/lib/pgsql/data"

# Build and install pgvector as root before switching users
WORKDIR /tmp
RUN git clone --depth 1 https://github.com/pgvector/pgvector.git \
    && cd pgvector && make && make install \
    && rm -rf /tmp/pgvector

RUN mkdir -p ${PGDATA} \
    && chown -R postgres:postgres /var/lib/pgsql \
    && chmod 700 ${PGDATA}

USER postgres
RUN initdb -D ${PGDATA} --auth-host=md5 --auth-local=trust

EXPOSE 5432
ENTRYPOINT ["postgres", "-D", "/var/lib/pgsql/data"]
```

Production Note

initdb at build time is acceptable for dev/test. For production, use an init container or Kubernetes Job to initialize PGDATA on the persistent volume at first boot, so data survives pod restarts cleanly.

main.tf (abridged)

```
resource "kubernetes_secret_v1" "pg" {
  metadata { name = "pg-secret"; namespace = var.namespace }
  data = {
    POSTGRES_PASSWORD = var.postgres_password  # Provider handles base64 — do NOT
wrap in base64encode()
  }
  type = "Opaque"
}

resource "kubernetes_stateful_set_v1" "pg" {
  metadata { name = "pgvector"; namespace = var.namespace }
  spec {
    service_name = "pgvector"
    replicas     = 1
    selector { match_labels = { app = "pgvector" } }
    template {
      metadata { labels = { app = "pgvector" } }
      spec {
        container {
          name  = "pg"
          image = "${var.image_name}:latest"
          env {
            name = "POSTGRES_PASSWORD"
```

```
            value_from {
              secret_key_ref {
                name = kubernetes_secret_v1.pg.metadata[0].name
                key  = "POSTGRES_PASSWORD"
              }
            }
          }
          port { container_port = 5432 }
          volume_mount { name = "pgdata"; mount_path = "/var/lib/pgsql/data" }
        }
      }
    }
    volume_claim_template {
      metadata { name = "pgdata" }
      spec {
        access_modes = ["ReadWriteOnce"]
        resources { requests = { storage = var.storage } }
      }
    }
  }
}

output "dsn" {
  value =
"postgresql://postgres:${var.postgres_password}@pgvector.${var.namespace}.svc.clust
er.local:5432/postgres"
}
```

A.4 Module: zosmf-workflow-trigger

Triggers z/OSMF workflow instances via REST API with RACF authentication. Includes idempotency guard and optional auto-start.

> **Key Design Note**
> *X-CSRF-ZOSMF-HEADER is required on all mutating z/OSMF REST calls since z/OSMF 2.3. Omitting it returns HTTP 403. This is the most common integration gotcha.*

providers.tf

```
terraform {
  required_providers {
    http = { source = "hashicorp/http", version = "~> 3.4" }
    null = { source = "hashicorp/null", version = "~> 3.2" }
  }
}
```

variables.tf (key variables)

```
variable "zosmf_base_url" {
  type        = string
  description = "z/OSMF REST base URL, e.g. https://zosmf.sysplex.local:443"
```

```
  validation {
    condition     = can(regex("^https://", var.zosmf_base_url))
    error_message = "zosmf_base_url must use HTTPS."
  }
}
variable "zosmf_user"          { type = string; sensitive = true }
variable "zosmf_password"       { type = string; sensitive = true }
variable "workflow_definition_path" { type = string }
variable "workflow_name"        { type = string }
variable "workflow_owner"       { type = string }
variable "system_name"          { type = string }
variable "workflow_variables"   { type = map(string); default = {} }
variable "auto_start" {
  type    = bool
  default = false  # Always false in prod — human must approve
}
```

main.tf

```
locals {
  all_headers = merge(
    { "Content-Type" = "application/json"; "X-CSRF-ZOSMF-HEADER" = "true" },
    { "Authorization" = "Basic
${base64encode("${var.zosmf_user}:${var.zosmf_password}")}" }
  )
  workflows_endpoint = "${var.zosmf_base_url}/zosmf/workflow/rest/1.0/workflows"
}

# Idempotency guard — check for existing instance before creating
data "http" "existing_workflows" {
  url    =
"${local.workflows_endpoint}?workflowName=${urlencode(var.workflow_name)}"
  method = "GET"
  request_headers = local.all_headers
  lifecycle {
    postcondition {
      condition     = contains([200], self.status_code)
      error_message = "z/OSMF query failed (HTTP ${self.status_code}). Check RACF
IZUDFLT access."
    }
  }
}

locals {
  existing_list   = jsondecode(data.http.existing_workflows.response_body)
  workflow_exists = length(try(local.existing_list.items, [])) > 0
  existing_key    = local.workflow_exists ?
local.existing_list.items[0].workflowKey : ""
}

resource "null_resource" "create_workflow" {
  count = local.workflow_exists ? 0 : 1
  triggers = {
    workflow_name   = var.workflow_name
    variables_hash  = sha256(jsonencode(var.workflow_variables))
  }
  provisioner "local-exec" {
    interpreter = ["/bin/bash", "-c"]
    command = <<-EOT
```

```
      RESPONSE=$(curl -s -w "\n%{http_code}" \
        -X POST "${local.workflows_endpoint}" \
        -H "Content-Type: application/json" \
        -H "X-CSRF-ZOSMF-HEADER: true" \
        -H "Authorization: Basic $(echo -n
'${var.zosmf_user}:${var.zosmf_password}' | base64)" \
        -d '{"workflowName":"${var.workflow_name}",
             "workflowDefinitionFile":"${var.workflow_definition_path}",
             "system":"${var.system_name}","owner":"${var.workflow_owner}",
             "assignToOwner":true}')
      HTTP_CODE=$(echo "$RESPONSE" | tail -1)
      BODY=$(echo "$RESPONSE" | head -n -1)
      [ "$HTTP_CODE" != "201" ] && echo "ERROR: $HTTP_CODE $BODY" && exit 1
      echo "$BODY" | python3 -c "import sys,json;
print(json.load(sys.stdin)['workflowKey'])" \
        > /tmp/zosmf_wf_${var.workflow_name}.txt
    EOT
  }
}

output "workflow_key" {
  value = local.workflow_exists ? local.existing_key : "pending-creation"
}
```

Appendix A.5: Agent Tool Manifest & Runtime Sketch

The tool_manifest.json.tpl file referenced in the agent-tool-catalog module defines the complete OpenAPI-style tool catalog that vLLM receives in every agent session. The manifest is rendered by Terraform at deploy time with the correct z/OSMF and CMCI endpoints, then mounted as a read-only ConfigMap into the agent pod.

A.5.1 files/tool_manifest.json.tpl

```
{
  "tools": [

    {
      "type": "function",
      "function": {
        "name": "get_job_status",
        "description": "Retrieve the current status and return code of a JES2 job
by name or job ID. Use this before any decision to cancel or resubmit a job.",
        "parameters": {
          "type": "object",
          "properties": {
            "job_id":   { "type": "string", "description": "JES2 job ID (e.g.
JOB01234). Provide either job_id or job_name." },
            "job_name": { "type": "string", "description": "Job name pattern (e.g.
PAYROLL*). Wildcards accepted." },
            "rationale":{ "type": "string", "description": "Plain English
explanation of why you are checking this job." }
          },
          "required": ["rationale"]
        }
      }
    },

    {
      "type": "function",
      "function": {
        "name": "list_active_address_spaces",
        "description": "List active z/OS address spaces with CPU%, real storage,
and extended storage usage. Use to diagnose performance issues before recommending
WLM changes.",
        "parameters": {
          "type": "object",
          "properties": {
            "filter_name": { "type": "string", "description": "Optional address
space name filter (e.g. CICS*)." },
            "top_n":       { "type": "integer", "description": "Return only the top
N by CPU usage. Default 20." },
            "rationale":   { "type": "string", "description": "Why you need this
information." }
          },
          "required": ["rationale"]
        }
      }
    },

    {
      "type": "function",
      "function": {
```

```
        "name": "get_dataset_catalog_entry",
        "description": "Look up a z/OS dataset in the catalog. Returns VOLSER,
DSORG, RECFM, LRECL, and allocation. Does not read dataset contents.",
        "parameters": {
          "type": "object",
          "properties": {
            "dsn":      { "type": "string", "description": "Fully qualified dataset
name. Wildcards accepted for listing." },
            "rationale":{ "type": "string" }
          },
          "required": ["dsn", "rationale"]
        }
      }
    },

    {
      "type": "function",
      "function": {
        "name": "get_wlm_policy",
        "description": "Retrieve the active WLM service definition and service
class goals. Use before recommending or requesting WLM policy changes.",
        "parameters": {
          "type": "object",
          "properties": {
            "rationale": { "type": "string" }
          },
          "required": ["rationale"]
        }
      }
    },

    {
      "type": "function",
      "function": {
        "name": "get_cics_region_stats",
        "description": "Retrieve CICS region statistics via CMCI: task counts,
transaction rates, DSA/EDSA usage, and active connections.",
        "parameters": {
          "type": "object",
          "properties": {
            "region_name": { "type": "string", "description": "CICS region applid
(e.g. CICSPROD)." },
            "rationale":   { "type": "string" }
          },
          "required": ["region_name", "rationale"]
        }
      }
    },

    {
      "type": "function",
      "function": {
        "name": "get_smf_summary",
        "description": "Retrieve a summary of recent SMF records of a given type
for a time window. Returns counts, averages, and anomaly flags. Does not return raw
PII fields.",
        "parameters": {
          "type": "object",
          "properties": {
            "smf_type":    { "type": "integer", "description": "SMF record type
(e.g. 110 for CICS, 30 for job accounting)." },
```

```
            "minutes_back":{ "type": "integer", "description": "How many minutes
back to summarize. Max 60." },
            "rationale":   { "type": "string" }
          },
          "required": ["smf_type", "minutes_back", "rationale"]
        }
      }
    },

    {
      "type": "function",
      "function": {
        "name": "analyze_syslog",
        "description": "Search the z/OS system log for messages matching a pattern
in a recent time window. Returns matching lines with timestamps.",
        "parameters": {
          "type": "object",
          "properties": {
            "pattern":     { "type": "string", "description": "Message ID prefix or
text pattern (e.g. IEA or ABEND0C4)." },
            "minutes_back":{ "type": "integer", "description": "How many minutes
back to search. Max 120." },
            "rationale":   { "type": "string" }
          },
          "required": ["pattern", "minutes_back", "rationale"]
        }
      }
    },

    {
      "type": "function",
      "function": {
        "name": "validate_jcl",
        "description": "Submit JCL for syntax validation only — no actual
execution. Returns parse errors and warnings.",
        "parameters": {
          "type": "object",
          "properties": {
            "jcl_text":  { "type": "string", "description": "The JCL to validate.
Maximum 64KB." },
            "rationale": { "type": "string" }
          },
          "required": ["jcl_text", "rationale"]
        }
      }
    },

    {
      "type": "function",
      "function": {
        "name": "submit_job",
        "description": "Submit a JCL job for execution. REQUIRES an approved z/OSMF
workflow ticket ID. Will not execute without approval_ticket_id.",
        "parameters": {
          "type": "object",
          "properties": {
            "jcl_text":          { "type": "string", "description": "The JCL to
submit." },
            "approval_ticket_id":{ "type": "string", "description": "Approved
z/OSMF workflow instance key. REQUIRED." },
            "rationale":         { "type": "string" }
          },
```

```
          "required": ["jcl_text", "approval_ticket_id", "rationale"]
        }
      }
    },

    {
      "type": "function",
      "function": {
        "name": "cancel_job",
        "description": "Cancel a running JES2 job. REQUIRES an approved z/OSMF
workflow ticket ID.",
        "parameters": {
          "type": "object",
          "properties": {
            "job_id":            { "type": "string" },
            "approval_ticket_id":{ "type": "string", "description": "REQUIRED." },
            "rationale":         { "type": "string" }
          },
          "required": ["job_id", "approval_ticket_id", "rationale"]
        }
      }
    },

    {
      "type": "function",
      "function": {
        "name": "trigger_workflow",
        "description": "Start an existing z/OSMF workflow instance by its workflow
key. Use to initiate approved runbooks.",
        "parameters": {
          "type": "object",
          "properties": {
            "workflow_key":      { "type": "string", "description": "z/OSMF
workflow instance key." },
            "approval_ticket_id":{ "type": "string", "description": "REQUIRED." },
            "rationale":         { "type": "string" }
          },
          "required": ["workflow_key", "approval_ticket_id", "rationale"]
        }
      }
    }

  ]
}
```

A.5.2 Agent Runtime: Agentic Loop Sketch

The following Python sketch shows the structure of the agent runtime. This is not a complete implementation, it illustrates the key control points: the blocked pattern check, the approval gate, and the SMF write on every tool execution. Your production runtime should be built on a framework such as LangGraph or a custom agentic loop, with these checks enforced at the framework level, not the prompt level.

```
# agent_runtime/agent.py  — structural sketch
import json, re, os, requests
from smf_writer import write_smf_record
```

```
ZOSMF_BASE    = os.environ["ZOSMF_BASE_URL"]
VLLM_ENDPOINT = os.environ["VLLM_ENDPOINT"]
MODEL_NAME    = os.environ["MODEL_NAME"]
MAX_ITER      = int(os.environ.get("MAX_ITERATIONS", "10"))
TOKEN_BUDGET  = int(os.environ.get("TOKEN_BUDGET", "8192"))

# Load policy files from mounted ConfigMaps
with open("/etc/agent/policy/blocked_patterns.json") as f:
    BLOCKED = json.load(f)["patterns"]

with open("/etc/agent/tools/tool_manifest.json") as f:
    TOOLS = json.load(f)["tools"]

with open("/etc/agent/prompts/system_prompt.txt") as f:
    SYSTEM_PROMPT = f.read()


def is_blocked(tool_name: str, arguments: dict) -> str | None:
    """Return block reason if any blocked pattern matches. None if clean."""
    text = json.dumps({"tool": tool_name, "args": arguments}).upper()
    for rule in BLOCKED:
        if re.search(rule["match"], text):
            return rule["reason"]
    return None


def requires_approval(tool_name: str) -> bool:
    WRITE_TOOLS = {"submit_job", "cancel_job", "set_wlm_policy",
                   "update_cics_urimap", "trigger_workflow"}
    return tool_name in WRITE_TOOLS


def verify_approval(ticket_id: str) -> bool:
    """Check z/OSMF workflow ticket status. Returns True if COMPLETED."""
    url = f"{ZOSMF_BASE}/zosmf/workflow/rest/1.0/workflows/{ticket_id}"
    resp = requests.get(url, headers=zosmf_headers(), verify=True)
    if resp.status_code != 200:
        return False
    data = resp.json()
    return data.get("statusName") == "complete"


def execute_tool(tool_name: str, arguments: dict,
                 session_id: str, racf_user: str) -> dict:
    """
    Dispatch a tool call through the three cage layers.
    Returns {"result": ..., "outcome": "EXECUTED"|"REJECTED"|"BLOCKED"}
    """
    # Layer 1 — blocked pattern check
    block_reason = is_blocked(tool_name, arguments)
    if block_reason:
        write_smf_record({
            "racf_userid": racf_user, "session_id": session_id,
            "tool_name": tool_name,
            "rationale": arguments.get("rationale", ""),
            "outcome": "BLOCKED", "token_count": 0,
        })
        return {"result": f"BLOCKED: {block_reason}", "outcome": "BLOCKED"}

    # Layer 2 — approval gate for write operations
    if requires_approval(tool_name):
```

```
        ticket = arguments.get("approval_ticket_id", "")
        if not ticket or not verify_approval(ticket):
            write_smf_record({
                "racf_userid": racf_user, "session_id": session_id,
                "tool_name": tool_name,
                "rationale": arguments.get("rationale", ""),
                "outcome": "REJECTED", "token_count": 0,
            })
            return {
                "result": "REJECTED: Write operation requires an approved "
                          "z/OSMF workflow ticket (approval_ticket_id).",
                "outcome": "REJECTED"
            }

    # Layer 3 — execute and audit
    result = dispatch_to_zosmf(tool_name, arguments)
    write_smf_record({
        "racf_userid": racf_user, "session_id": session_id,
        "tool_name": tool_name,
        "rationale": arguments.get("rationale", ""),
        "approved_by": arguments.get("approval_ticket_id", "N/A"),
        "outcome": "EXECUTED", "token_count": 0,
    })
    return {"result": result, "outcome": "EXECUTED"}


def run_session(user_prompt: str, session_id: str,
                racf_user: str) -> str:
    """Run a bounded agentic loop. Returns final answer."""
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user",   "content": user_prompt},
    ]
    total_tokens = 0

    for iteration in range(MAX_ITER):
        resp = requests.post(
            f"{VLLM_ENDPOINT}/v1/chat/completions",
            json={"model": MODEL_NAME, "messages": messages,
                  "tools": TOOLS, "tool_choice": "auto",
                  "max_tokens": min(1024, TOKEN_BUDGET - total_tokens)},
            timeout=120
        )
        resp.raise_for_status()
        data = resp.json()
        total_tokens += data["usage"]["total_tokens"]
        choice = data["choices"][0]

        if choice["finish_reason"] == "stop":
            return choice["message"]["content"]  # Final answer

        if choice["finish_reason"] == "tool_calls":
            tool_calls = choice["message"]["tool_calls"]
            messages.append(choice["message"])
            for tc in tool_calls:
                args = json.loads(tc["function"]["arguments"])
                result = execute_tool(
                    tc["function"]["name"], args, session_id, racf_user
                )
                messages.append({
                    "role": "tool",
                    "tool_call_id": tc["id"],
```

```
                    "content": json.dumps(result)
                })

        if total_tokens >= TOKEN_BUDGET:
            return "Token budget exhausted. Partial findings: see SMF log."

    return "Maximum iterations reached. Summarize findings from SMF log."
```

Implementation Note

The dispatch_to_zosmf() function and zosmf_headers() are left as integration points for your environment. They should call the z/OSMF REST endpoints using the Kerberos-authenticated session from Chapter 5.5, not a stored password. The sketch above shows control flow only authentication is handled at the HTTP layer below it.

Appendix B: COBOL, JCL, and Integration Samples

B.1 CICS COBOL: Call vLLM and Parse JSON Response

This example uses EXEC CICS WEB to POST JSON to vLLM's OpenAI-compatible endpoint and uses Enterprise COBOL JSON PARSE to extract the response. The NAME OF phrase maps dotted JSON paths to COBOL data names.

```
IDENTIFICATION DIVISION.
       PROGRAM-ID. AIINFER1.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01  WS-URL        PIC X(256)
           VALUE "https://vllm-ai.apps.z.local/v1/chat/completions".
       01  WS-HEAD-CT    PIC X(20)  VALUE "Content-Type".
       01  WS-HEAD-VAL   PIC X(40)  VALUE "application/json".

      *> Request body — vLLM OpenAI chat format
       01  WS-REQ        PIC X(2048) VALUE
           '{"model":"ibm-granite","messages":[{"role":"user",'
           '"content":"Summarize claim 1234."}],"max_tokens":256}'.

       01  WS-RESP       PIC X(8192).
       01  WS-RESP-LEN   PIC S9(9) COMP VALUE 0.
       01  SESSTOKEN     PIC X(16).

      *> JSON PARSE target structure
      *> vLLM returns: {"choices":[{"message":{"content":"..."}}]}
       01  WS-AI-RESP.
           05 WS-CHOICES OCCURS 1 TIMES.
              10 WS-MESSAGE.
                 15 WS-CONTENT  PIC X(2048).

       01  WS-PARSE-OK   PIC S9(9) COMP VALUE 0.
       01  WS-ERR-MSG    PIC X(80).

       PROCEDURE DIVISION.
       MAIN-LOGIC.
           EXEC CICS WEB OPEN
                URIMAP('AIVLLM')
                SESSTOKEN(SESSTOKEN)
           END-EXEC.

           EXEC CICS WEB WRITE HTTPHEADER
                NAME(WS-HEAD-CT)   NAMELENGTH(12)
                VALUE(WS-HEAD-VAL) VALUELENGTH(16)
                SESSTOKEN(SESSTOKEN)
           END-EXEC.

           EXEC CICS WEB SEND
                FROM(WS-REQ)
                FROMLENGTH(LENGTH OF WS-REQ)
                MEDIATYPE('application/json')
                METHOD(POST)
                SESSTOKEN(SESSTOKEN)
           END-EXEC.

           EXEC CICS WEB RECEIVE
                INTO(WS-RESP)
```

```
                LENGTH(WS-RESP-LEN)
                SESSTOKEN(SESSTOKEN)
           END-EXEC.

           EXEC CICS WEB CLOSE SESSTOKEN(SESSTOKEN) END-EXEC.

      *> NAME OF maps JSON path to COBOL data name
           JSON PARSE WS-RESP
                INTO WS-AI-RESP
                WITH DETAIL
                NAME OF WS-CONTENT IS 'choices(1).message.content'
           ON EXCEPTION
                MOVE FUNCTION EXCEPTION-STATUS TO WS-ERR-MSG
                EXEC CICS WRITE OPERATOR
                     TEXT(WS-ERR-MSG) TEXTLENGTH(80)
                END-EXEC
                MOVE 1 TO WS-PARSE-OK
           NOT ON EXCEPTION
                MOVE 0 TO WS-PARSE-OK
           END-JSON.

           IF WS-PARSE-OK NOT = 0
              EXEC CICS RETURN END-EXEC
           END-IF.

           *> WS-CONTENT now holds the model reply
           GOBACK.
```

Implementation Note

Define the URIMAP AIVLLM with TLS enabled and connection pooling. The URIMAP defines the base URL; the EXEC CICS WEB SEND specifies the path. Use z/OS Connect EE requester mode for generated stubs if you prefer not to manage HTTP manually.

B.2 JCL: Batch RAG Ingestion via BPXBATCH + curl

```
//INGEST  JOB (ACCT),'RAG LOAD',CLASS=A,MSGCLASS=X,NOTIFY=&SYSUID
//STEP1   EXEC PGM=BPXBATCH,REGION=0M,
//  PARM='SH curl -sS -X POST https://rag.ai.apps.z.local/ingest
//        -H "Content-Type: application/json"
//        --data @/u/ingest/new_docs.json'
//STDOUT  DD SYSOUT=*
//STDERR  DD SYSOUT=*
```

B.3 z/OS Connect EE: z/OS App as API Requester

Generate COBOL stubs from an OpenAPI specification using the z/OS Connect Build Toolkit. The generated stubs allow CICS, IMS, or batch programs to call REST APIs without any manual HTTP coding. This is the preferred pattern for production integrations where an OpenAPI spec already exists for the AI service.

Appendix C — Observability Correlation Model

The following diagram shows how CICS transaction IDs are correlated through the AI stack to produce a complete observability chain. Any break in this chain means you cannot diagnose a semantic failure.

```
flowchart LR
  A[CICS Txn ID] --> B[SMF 110 Subtype=1]
  A --> C[AI Gateway Log]
  C --> D[Prompt ID]
  D --> E[Vector Query p95]
  D --> F[Model Latency p95]
  E --> G{SLO}
  F --> G
```

SMF Type	Source	AI Correlation Field
110 Subtype 1	CICS monitoring	Transaction ID → Prompt ID
110 Subtype 99	AI agent runtime	Session ID → Tool name → Outcome
30	Job/task accounting	Batch ingest job → RAG load timing
42	Address space activity	vLLM pod CPU/memory vs. inference latency

Appendix D: Implementation References

IBM Z Hardware and AI Acceleration

- IBM z16 / Telum on-chip AI, real-time fraud/analytics, low-latency inference: ibm.com/z16
- Telum II & Spyre (z17/LinuxONE 5), scaled LLM inferencing, DPU I/O: ibm.com/z17
- IBM AI Toolkit for Z & LinuxONE, optimized TF/PyTorch/ONNX: ibm.com/products/ai-toolkit-z-linuxone

OpenShift on IBM Z

- OpenShift on IBM Z & LinuxONE install docs (UPI / disconnected): docs.redhat.com
- Red Hat OpenShift Container Platform s390x release notes: docs.redhat.com

z/OS Integration

- z/OS Connect EE provider and requester modes: ibm.com/docs/zos-connect
- CICS WEB commands (OPEN/WRITE/SEND/RECEIVE/CLOSE): ibm.com/docs/cics-ts
- Enterprise COBOL JSON PARSE/GENERATE: ibm.com/docs/cobol-zos
- RACF Security fundamentals: ibm.com/docs/zos/racf
- SMF type 110 monitoring subtypes for CICS: ibm.com/docs/cics-ts/smf
- OMEGAMON for z/OS monitoring: ibm.com/docs/omegamon
- z/OSMF REST and Workflow APIs: ibm.com/docs/zosmf

Disaster Recovery

- GDPS (Geographically Dispersed Parallel Sysplex): ibm.com/gdps
- Coupling Facility architecture: redbooks.ibm.com — SG24-7901

Next Steps to Productionize

- Select target model: 7B-class models work well for CPU-only inference on s390x. Swap to IBM AI Toolkit runtimes for Telum-accelerated ONNX inference.
- Deploy in order: pgvector-on-ocp → openshift-vllm → smoke test /v1/chat/completions.
- Wire CICS URIMAP (TLS, pooled connections) and test the COBOL sample end-to-end.
- Enable SMF 110 monitoring and configure the AI observability dashboard.
- Add AI-BOM exporter: a CronJob that reads Terraform state and the live cluster API and emits the JSON template from Chapter 12.
- Configure z/OSMF approval workflows before enabling any agent write tools.

— End of Document —